ELEC327 Final Project - Cubington

May 15, 2026

Zak Hashmi, Milan Addepalli, Kasim Dawood, Nathan Cao

A lab report detailing the different
components and processes for the final project

Rice University ECE

Contents

1 Introduction

Our project involves creating a joystick utilizing a Hall effect-based joystick to output direction (horizontal and vertical) signals to the MSPM0. The signals are read and interpreted by using the ADC module. These values will subsequently be mapped to a 2D direction, which is then used as an input to an Adafruit display (this particular screen communicates with the MSPM0 through SPI).

As a high-level overview, our project will consist of the following components (there are additional parts to this project, but specifically these are central to our initial goals):

Custom-designed PCB
Sparkfun Electronics Analog Switch Joystick
Adafruit TFT LCD Touchscreen

The basic premise of the game is that the screen will provide a random orientation of a cube. The user is able to control another cube displayed on the screen and the goal is to match the geometry orientation of cube number 2 with the provided cube. The goal of the game is to match as many orientations as possible within a minute.

2 Joystick

The joystick is one of the core components of the game. The user uses the horizontal and vertical axes of the joystick to orient the cube provided on the screen. Shown below is a real-life representation of the joystick.

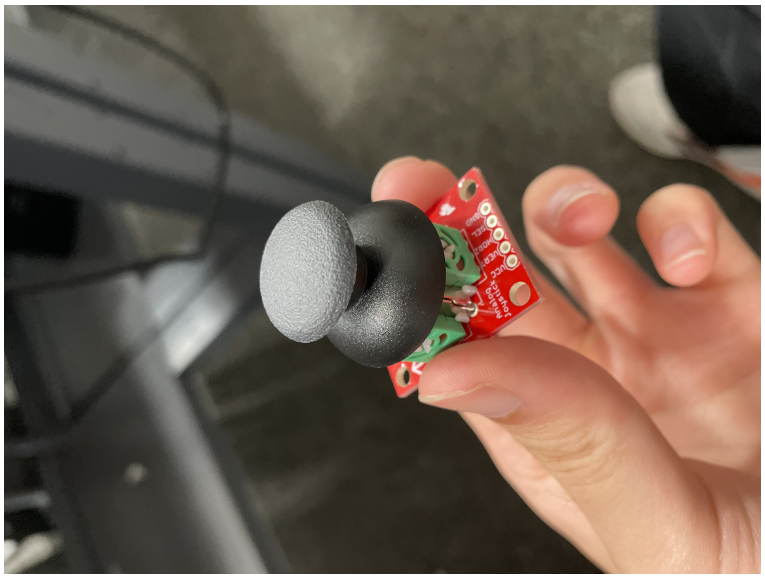

Figure 1: Physical Joystick

2.1 Physical Design Considerations

The joystick has 3 core components:

- Horizontal axis
- Vertical axis
- Button

Each axis works like a potentiometer, meaning it behaves like a variable voltage divider. The joystick is connected between **VDD** and **GND**, and as the stick moves, the wiper output for each axis changes voltage depending on its position. Those analog voltages are sent to the MSPM0 ADC pins, where the microcontroller ocnverts them into digital values that can be used in software to detect joystick direction and movement.

In order to yield stable voltage values and smooth out noise, we considered adding capacitors from the joystick signal lines to ground. However, we decided not to include them because the joystick movement does not require

extremely precise analog filtering, and the small amount of noise can be handled in software.

The specific pins and their functions will be discussed more in the PCB design section of this report.

2.2 Code Implementation

2.2.1 Initialization

The joystick X and Y axes act like potentiometers, outputting voltages between **GND** and **VDD**. The code configures two MSPM0 pins as analog inputs and uses `ADC0` to repeatedly sample these voltages.

2.2.2 Update Joystick State

The ADC samples both joystick channels in sequence. `joystick_update()` waits for the second result, stores both readings in `joystick_raw`, and `joystick_x()` / `joystick_y()` subtract 128 to center the values around zero. **DL_ADC12** is TI's DriverLib interface for the MSPM0 ADC.

2.2.3 Determining Direction

The code uses `JOYSTICK_DEADZONE` to ignore small center noise. If the joystick moves outside this range, it compares the X and Y values and returns the dominant direction. `joystick_magnitude()` computes the distance from center.

2.2.4 Button Logic

The joystick button is a digital active-low input with an internal pull-up resistor. It normally reads high, but pressing the button shorts the pin to ground, so `joystick_buttonPressed()` checks whether the pin reads 0.

3 Adafruit Screen

3.1 Physical Design Considerations

The screen is a core component of the project - it displays our game and also takes inputs from the user. Shown below is a real-life representation of the screen we use:

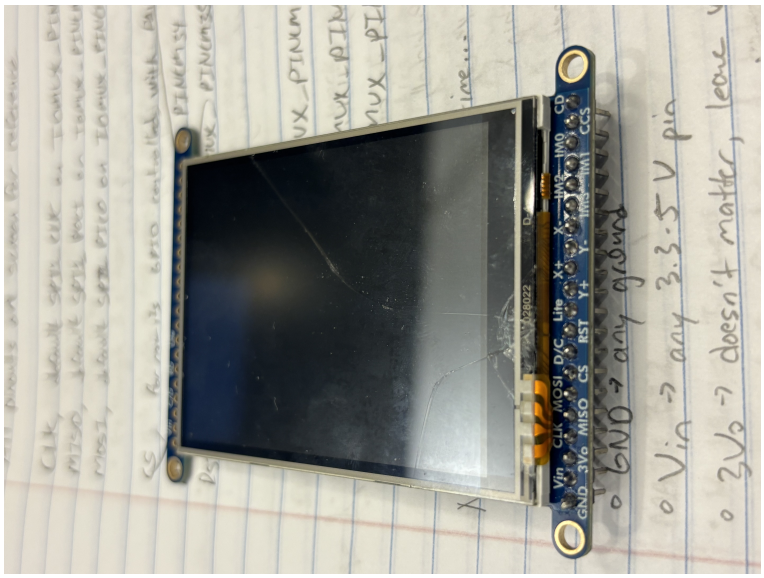

Figure 2: Physical Adafruit LCD Screen

The screen has a few main components:

- **CLK** which controls data transfer to the screen
- **MOSI** which is data input to the screen

- **MISO** which sends data back to the microcontroller

Note that touchscreen is not being used, and the other pins and more screen functionality will be discussed later on in the PCB design section of this report.

3.2 Code Implementation

The screen code uses a compact framebuffer system to make drawing efficient on the MSPM0. Instead of storing the full 240 x 320 LCD image directly, the display is treated as a 120 x 160 logical screen where each 1-bit pixel represents either black or white and is scaled into a 2 x 2 block on the physical LCD. The `screen_buffer` stores this packed bitmap, while `Screen_SetPixel()` and `Screen_GetPixel()` convert coordinates into byte and bit positions to save RAM. The LCD is initialized in `drawmethods.c` with commands for reset, orientation, pixel format, sleep-out, and display-on, after which `DrawBitmap()` streams RGB565 pixel data over SPI. To improve speed, `Screen_Display()` compares the current buffer with `prev_buffer`, finds the smallest changed region, and uses `DrawBitmapRegion()` to redraw only that area. The code also includes a `scratch_buffer` for temporary offscreen rendering before copying or shifting objects into the main framebuffer.

4 PCB Design

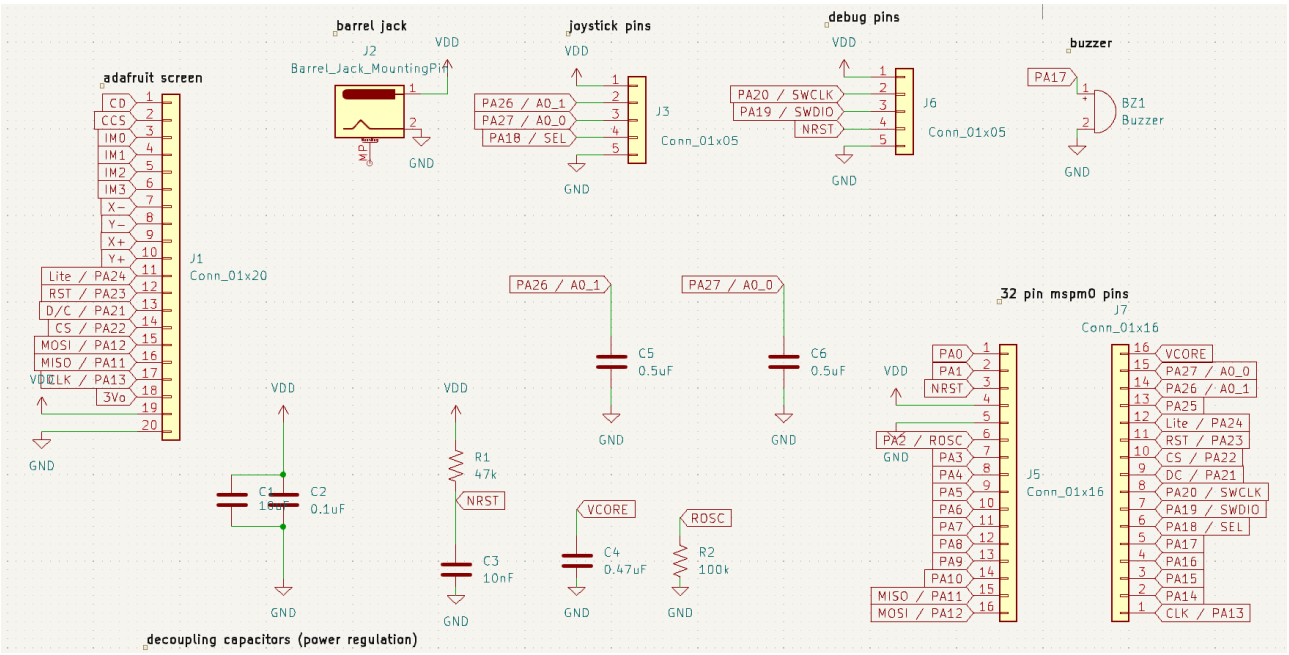

Figure 3: Custom PCB Schematic

4.1 Adafruit LCD Screen

As a brief overview, we describe each connection we use for the Adafruit screen. We knew initially that we will not be using the touchscreen functionaliy, but there needs to be an output display and it needs to take an input as well from the joystick.

- **Vin** takes a 3.3V input from the barrel jack and provides power to the screen
- **GND** is the ground reference.
- **CLK** is the SPI clock and is used to regulate/toggle data transfer to the screen.
- **MOSI** is SPI data input to the display. This is how pixel data and commands are sent, and how our joystick communicates with the screen.
- **MISO** is the data line that lets the peripheral send data back to the microcontroller

- **CS** is chip select, and this lets the screen know when a GPI0 pin is sending input.
- **D/C** is data/command select. Tells the screen whether the SPI byte is a command or display data.
- **RST** resets the display controller.
- **Lite** is the backlight control pin for the screen.

4.2 Other Components

The PCB uses a **barrel jack** to supply 3.3V power to the MSPM0 and the rest of the board. At the center of the design is the **32-pin QFN MSPM0**, which connects the power, reset, debug, screen, joystick, and other I/O signals. The **joystick** provides **VDD**, **GND**, **SEL**, and two potentiometer-like analog outputs that are read by ADC pins PA26 and PA27; we left these analog lines without capacitors because software dead zones and averaging can handle small noise while keeping the response fast.

For programming and reliability, the board also includes a **debug header**, **decoupling capacitors**, and required MSPM0 support circuitry. The debug header exposes **PA20/SWCLK**, **PA19/SWDIO**, **NRST**, **VDD**, and **GND** so an external probe can flash and debug the chip. The decoupling capacitors stabilize **VDD**, the **NRST** resistor/capacitor network provides reliable startup and debugger-controlled reset, the **VCORE** capacitor stabilizes the internal core voltage, and the **ROSC** resistor supports the oscillator/reference circuit.

5 Conclusion

In conclusion, we have created Cubington, a game where the user controls a Hall effect-based joystick to output direction (horizontal and vertical) and orient a cube in a specific way. For the display, we used an Adafruit LCD Screen (SPI communication protocol) taking inputs from the joystick.

6 Additional Notes (Contributions)

Zak Hashmi - 35%
Milan Addepalli - 30%
Nathan Cao - 30%
Kasim Dawood - 5%

Zak Hashmi: Created all screen code and game code. Created GitHub repository and website. Final Prototyping. Team Lead.

Milan Addepall: Created all other peripheral code. Final prototyping. Demo video.

Nathan Cao: Created PCB. Final Report. Final prototyping.

Kasim Dawood: Attempt at PCB. Attempt at Final Report.